

Echoes — Interview Revision Guide

One-liner (30 seconds)

Echoes is a location-aware mobile travel app I built with **React Native (Expo)** and a **Node.js/Express** backend. It uses the user's GPS to find nearby historical landmarks via the **Google Places API**, generates rich AI narratives with **OpenAI GPT-4**, reads them aloud with **text-to-speech**, and gamifies exploration through quizzes, badges, and a leaderboard.

Problem

Tourists and history enthusiasts often visit landmarks without context. Guidebooks are static, tour guides aren't always available, and most map apps show *where* something is — not *why it matters*. I wanted to turn passive sightseeing into an interactive, educational experience.

What I Built

A full-stack mobile app with:

- GPS-based landmark discovery** — detects the user's location and fetches nearby tourist attractions
- AI storytelling** — GPT-4 writes detailed historical narratives for each landmark (or city, as fallback)
- Voice narration** — expo-speech reads stories aloud for a hands-free, immersive experience
- User accounts & persistence** — JWT auth, MongoDB stores visited places and scores
- Gamification** — dynamic quizzes generated from places you've visited, scoring system, leaderboard
- Profile dashboard** — tracks exploration history, score, and visited landmarks

Tech Stack

Layer	Technologies
Frontend	React Native, Expo 54, TypeScript, React Navigation
Location & Media	expo-location, expo-speech, Google Places & Geocoding APIs
AI	OpenAI GPT-4 (narratives), GPT-3.5 (quiz generation)
Backend	Node.js, Express, MongoDB (Mongoose)
Auth	JWT, bcrypt, AsyncStorage
Networking	Axios, REST APIs

Architecture / Data Flow

User opens app

- Login/Signup (JWT stored in AsyncStorage)
- MapScreen requests location permission

- GPS coordinates sent to Google Places Nearby Search
- Filter for historical sites (museums, forts, tombs, etc.)
- Haversine formula finds nearest landmark
- Landmark name sent to OpenAI → narrative generated
- Story displayed + read aloud via TTS
- User marks place as visited → saved to MongoDB
- Quiz screen pulls visited places → GPT generates MCQ quiz
- Score updated → reflected on Profile & Leaderboard

Key Features to Explain in Depth

1. Landmark Detection

- Uses Google Places API with a 5km radius
- Filters results by type (historical_site, museum, fort, etc.)
- Custom **Haversine distance** calculation to pick the closest landmark
- Fallback: if no historical sites found, reverse-geocodes coords to city name and generates a city-level story

2. AI Narrative Generation

- Prompt-engineered for travel-writer tone (8–10 paragraphs)
- Separate prompts for monuments vs. cities
- Handles loading states and API failures gracefully

3. Gamification Loop

- Visiting a place → stored in `placesVisited[]` on User model
- Quiz dynamically generated from *your* visited places (personalized learning)
- Scoring: +100 per correct answer, –20 per wrong answer
- Leaderboard ranks users by total score

4. Auth & Backend

- REST API: `/api/auth`, `/api/user`, `/api/quiz`
- Password hashing with bcrypt
- JWT middleware for protected routes
- MongoDB schema: User with nested `placesVisited` and `score`

Interview Talking Points (STAR-style)

"Tell me about a technical challenge you faced."

Getting accurate landmark data was tricky — Google Places returns many generic tourist spots. I solved this by filtering on specific place types and implementing a nearest-neighbor algorithm using the Haversine formula. When no historical sites were nearby, I added a reverse-geocoding fallback so the app still delivered value.

"How did you integrate AI?"

I used OpenAI's Chat Completions API in two ways: GPT-4 for long-form landmark narratives and GPT-3.5 for structured quiz JSON. For quizzes, I had to parse GPT's free-text response with regex to extract valid JSON — a real-world lesson in making LLM outputs reliable.

"What would you improve?"

Move API keys to environment variables and a backend proxy (they're currently client-side). Add caching for generated stories to reduce API costs. Use EAS Build for production APKs. Add offline support with cached narratives. Implement proper error boundaries and retry logic for network failures.

"Why React Native / Expo?"

Cross-platform from one codebase (iOS + Android). Expo simplified location, speech, and build tooling. React Navigation gave a clean stack-based flow from login → explore → quiz → profile.

Screens / Modules

Screen	Purpose
Login / Signup	Auth flow
HistoryExplorer	Home hub — navigate to map, quiz, leaderboard
MapScreen	Core feature — location, landmarks, AI stories, TTS
PlacesVisited	Log of explored landmarks
Quiz	AI-generated quiz from visited places
Leaderboard	Competitive ranking
Profile	User stats, score, logout

Resume Bullet Versions

- Built **Echoes**, an AI-powered landmark exploration app using React Native, Google Maps API, and OpenAI GPT-4 for location-based historical storytelling with voice narration.
- Designed a **Node.js/Express + MongoDB** backend with JWT authentication, user progress tracking, and a gamified quiz system.
- Implemented **GPS landmark detection** with Haversine distance calculation and intelligent fallback to city-level content.

Likely Follow-up Questions — Quick Answers

Question	Answer
How does auth work?	Signup hashes password with bcrypt → JWT issued → stored in AsyncStorage → sent in Authorization header
How is nearest landmark calculated?	Haversine formula on lat/lng of user vs. each landmark
What if GPS is denied?	Permission check on Android + Expo Location; show error state

How are quizzes personalized?	Backend returns user's <code>placesVisited</code> ; GPT prompt includes those place names
Database schema?	User: <code>username</code> , <code>email</code> , <code>password</code> , <code>placesVisited[]</code> , <code>score</code>

Echoes — Vittesh Arora / vittesharora04@gmail.com / github.com/aroravittesh